Lecture 2

Rational Agents
State Space Representations
(Uninformed) Search Algorithms

Intelligent Systems
Vrije Universiteit

Today's lecture is an introduction to solving state-space representations by search, more precisely by uninformed search, and how they're used for decision making in AI.

Overview

1. Relational Agents
2. Task Environments
3. State Space Representations
4. Search Algorithms
5. Uninformed Search
6. Search Directions

Part 1: Rational Agents

Intelligent Systems
Vrije Universiteit

The following section concerns state space representations, which can be used to abstract real environments.

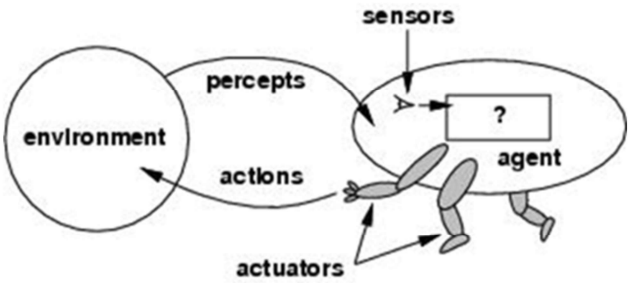
Agents

The *agent function* maps percept sequence to actions

$$f : P^* \rightarrow A$$

The agent function will internally be represented by the *agent program*.

The agent program runs on the physical *architecture* to produce *f*.



4

First let us reflect on the notion of an Agent, which is central to this course.

Here we choose a simple technical definition of agents: some entity (the “agent”) which consists of a set of sensors and actuators.With the sensors the agent perceives the environment, and with actuators it can manipulate the environment through a set of actions.

There are various levels of abstraction one can study agents,

1. functional, in other words agent behaviour as a function with sensor information as input (e.g. as a vector of features) and a (possibly complex) action as output.
2. as a program, in other words as the implementation in some system, and finally
3. w.r.t the specific architecture in which the agent operates.

These definitions are very general, so take some examples: the most simple would be an agent to regulate this rooms heating automatically. The agent function would determine to increase the heating in case the temperature gets below a certain threshold, the environment would be the physical environment of this lecture hall with the specific technical device, and the program would be the actual implementation. Of course, this would not be a very intelligent agent. On the other extreme, consider Grace, which operates in an open environment, the function is very complex as will be the actual implementation.

Rationality

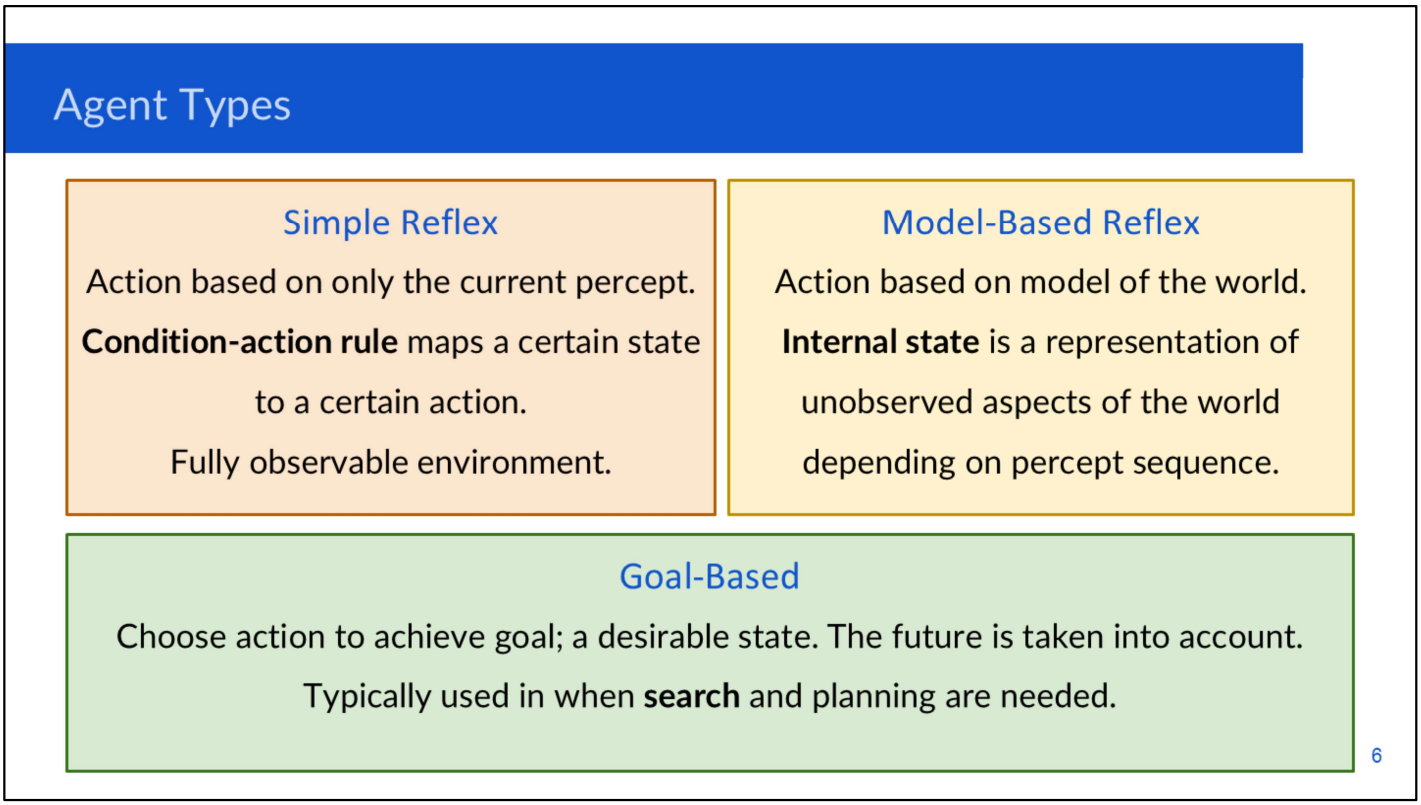
A rational agent chooses whichever action maximizes the expected value of the performance measure given the percept sequence to date and prior environment knowledge

What is rational at a given time depends:

- Expected value of performance measure (heuristics)
- Actions and choices (Search)
- Prior environment knowledge, (KR)
- Percept sequence to date (Learning)

Rationality is not the same as omniscience

Rationality is not the same as perfection



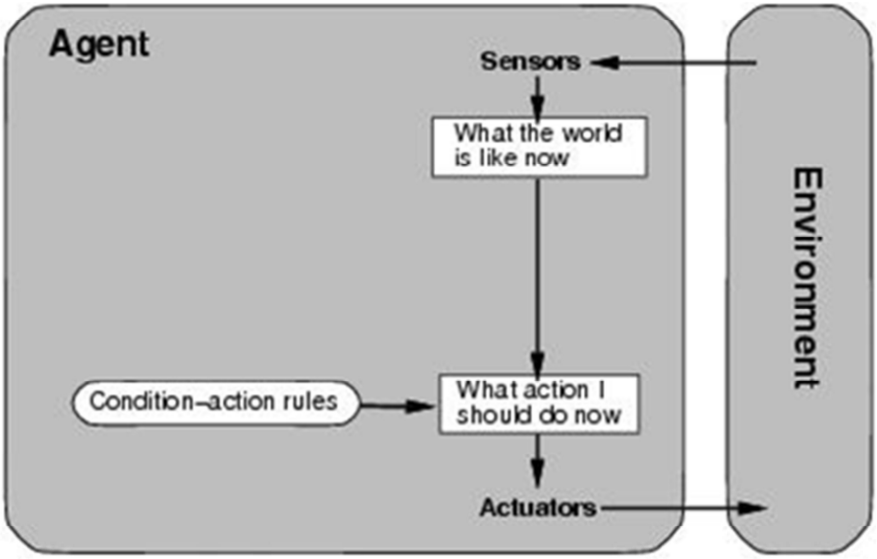
Agents vary in the complexity with which they make decisions.

The **simple reflex agent** will always make the same action given the same percept, decided by a **condition-action rule**; think of this as a type of *if… then* statement.

Model-based reflex agents are more complex, creating an **internal state** in order to deal with aspects of the environment that are not observable. This internal state is a kind of model, or representation, the agent builds using its prior knowledge.

Goal-based agents are the most complex; these agents play a long game in which they reason further into the future in order to choose an action. For example, they may use heuristics to choose the action with the least distance to the goal.

Agent types; simple reflex



7

Select action on the basis of *only the current* percept.

Implemented through *condition-action rules.*

Here's a pseudo-code for the agent function:

function REFLEX-AGENT-WITH-STATE(*percept*) **returns** an action

static: *rules*, a set of condition-action rules

state, a description of the current world state

action, the most recent action.

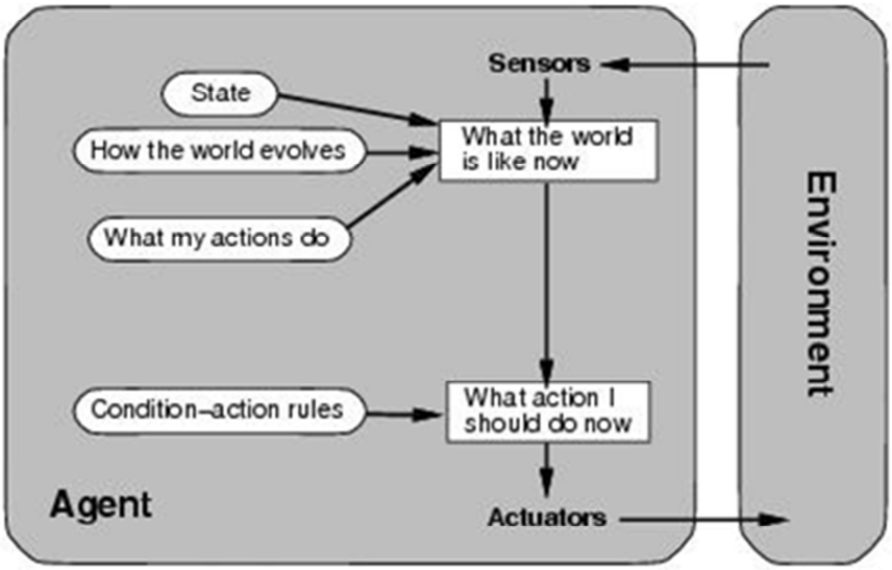
state ¬ UPDATE-STATE(*state*, *action*, *percept*)

rule ¬ RULE-MATCH(*state*, *rule*)

action ¬ RULE-ACTION[*rule*]

return *action*

Agent types: model based; reflex and state



8

To tackle *partially observable* environments.

– Maintain internal state

Over time update state using world knowledge

– How does the world change.

– How do actions affect world. *Model of World*

function REFLEX-AGENT-WITH-STATE(*percept*) **returns** an action

static: *rules*, a set of condition-action rules

state, a description of the current world state

action, the most recent action.

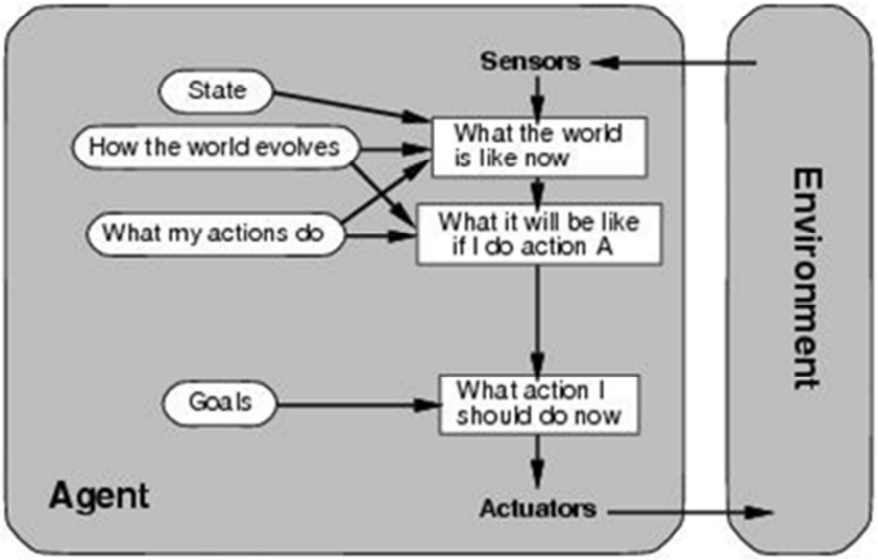
state ¬ UPDATE-STATE(*state*, *action*, *percept*)

rule ¬ RULE-MATCH(*state*, *rule*)

action ¬ RULE-ACTION[*rule*]

return *action*

Agent types: goal-based

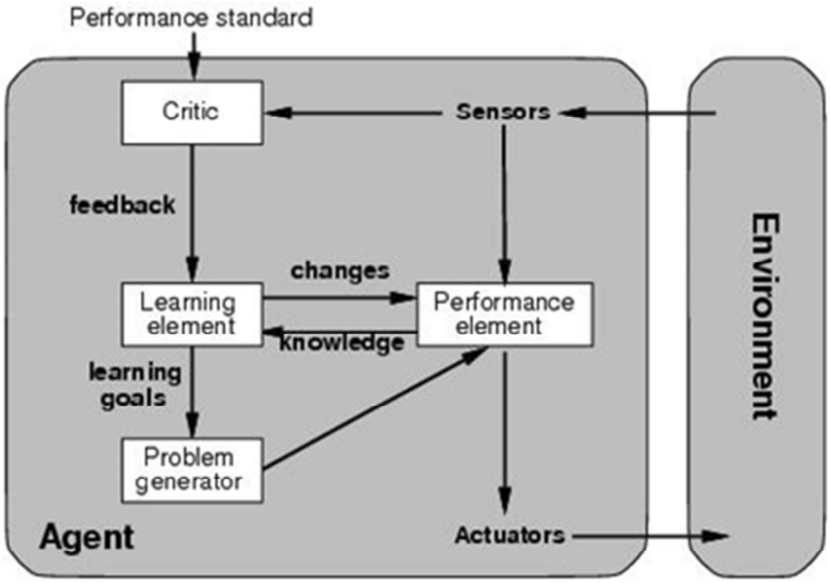


The agent needs a goal to know which situations are *desirable*.

Typically investigated in **search** and **planning** research.

Major difference: future is taken into account

Agent types: learning-based



10

All previous agent-programs describe methods for selecting *actions*.

– Yet it does not explain the origin of these programs.

– Learning mechanisms can be used to perform this task.

– Teach them instead of instructing them.

– Advantage is the robustness of the program toward initially unknown environments.

Part 2: Task Environments

Intelligent Systems
Vrije Universiteit

The following section concerns state space representations, which can be used to abstract real environments.

Task Environment

How do we characterize the problem that the agent must solve?

1. Performance: what it does
2. Environment: where it does it
3. Actuators: how it does it
4. Sensors: what it perceives

Acronym for task environment: PEAS

We can formalise the task environment with the acronym **PEAS**. To define a problem, we consider the agent's performance, environment, actuators, and sensors.

Environment Properties (not exhaustive)

Fully vs. Partially Observable	Sensors can detect all or some aspects relevant to an action.
Deterministic vs. Stochastic	The next environment state is completely determined by current state and executed action vs. other factors also at play.
Dynamic vs. Static	Environment can change while agent is choosing an action vs. no change.
Single vs. Multi-Agent	Whether the environment contains agents who are also maximising some performance measure dependent on current agent's actions.




13

The properties of an environment influence the way the agent perceives it and can take action.

In a fully **observable** environment, agents can perceive every element in the environment that may be relevant to the action, such as chess. In a **deterministic** environment, each action results in a predictable next set of possible actions, while **stochastic** environment involves an element of chance, for example rolling of dice.

Dynamic environments can change while the agent is deciding on its next action; e.g. a self-driving car has to adapt its actions to a dynamically changing environment with infinite possible changes. In **static** environments like chess there is no change. **Single** or **multi-agent** refers to the number of different agents who are also taking action in the same environment.

Environment Properties (not exhaustive)

		Schnapsen
Fully vs. Partially Observable	Sensors can detect all or some aspects relevant to an action.	Partial
Deterministic vs. Stochastic	The next environment state is completely determined by current state and executed action vs. other factors also at play.	Deter-ministic
Dynamic vs. Static	Environment can change while agent is choosing an action vs. no change.	Static
Single vs. Multi-Agent	Whether the environment contains agents who are also maximising some performance measure dependent on current agent's actions.	Multi-agent

Schnapsen has a partially observable environment because the agent can not see the full deck of cards in Phase 1. It is a deterministic environment because there are no elements that change other than the actions of the players. Schnapsen is static because there are distinct turns for each player, during which no other changes are made to the current state. It is a multi-agent game because there are multiple players each trying to reach their own goals.

Task Environments

To design a rational agent we must specify its task environment:

- Performance (what it does)
- Environment (where it does it)
- Actuators (how it does it)
- Sensors (what it perceives)

Remember PEAS acronym for task environment

15

A very important aspect of building intelligent, rational, agents is to understand the requirements for the actions of the agent that are determined by the environment an agent operates in. Imagine the simple agent that switches the heating of this room on and off. It is, of course, crucial to be fully aware of what the agent is supposed to do, where it is supposed to be doing it, how it can do it, and how it can perceive the world.

An agent that has only one sensor (e.g. the temperature in front of the room), will have to be designed in a different way than an agent that also measures temperature outside, or various sensors in the room, or including the number of people, or the time of day, etc.

So when building an Intelligent Agent, one has to analyse carefully the aspects of Performance, Environment, Actuators and Sensors, often abbreviated as PEAS criteria.

Let us briefly zoom into different environment types, as there is a close relationship between those types and the types of solutions we will discuss in the next few weeks.

Environment types

	Schnapsiance	Backgammon	Autonomous Car	Schnapsen
Fully-Observable	FULL	FULL	NO	??
Deterministic	YES	NO	NO	??
Static	YES	YES	NO	??
Discrete	YES	YES	NO	??
Single-agent	YES	NO	NO	??

16

Fully vs. partially observable: an environment is fully observable when the sensors can detect all aspects that are relevant to the choice of action.
Deterministic vs. stochastic: if the next environment state is completely determined by the current state & the executed action then the environment is deterministic.
Static vs. dynamic: If the environment can change while the agent is choosing an action, the environment is dynamic. Semi-dynamic if the agent's performance changes even when the environment remains the same.
Discrete vs Continuous: discrete information if there are only a finite number of values possible (count), Continuous information are not restricted to defined separate values (measure)
Single vs. multi-agent: Does the environment contain other agents who are also maximizing some performance measure that depends on the current agent's actions?

Environment types

	Schnapsiance	Backgammon	Autonomous Car	Schnapsen
Fully-Observable	FULL	FULL	NO	NO
Deterministic	YES	NO	NO	YES
Static	YES	YES	NO	YES
Discrete	YES	YES	NO	YES
Single-agent	YES	NO	NO	NO

17

Fully vs. partially observable: an environment is fully observable when the sensors can detect all aspects that are relevant to the choice of action.
Deterministic vs. stochastic: if the next environment state is completely determined by the current state & the executed action then the environment is deterministic.
Static vs. dynamic: If the environment can change while the agent is choosing an action, the environment is dynamic. Semi-dynamic if the agent's performance changes even when the environment remains the same.
Discrete vs Continuous: discrete information if there are only a finite number of values possible (count), Continuous information are not restricted to defined separate values (measure)
Single vs. multi-agent: Does the environment contain other agents who are also maximizing some performance measure that depends on the current agent's actions?

The following section concerns state space representations, which can be used to abstract real environments.

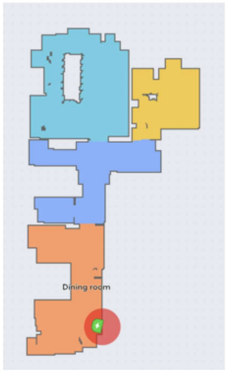
One of the main features of the discipline of Intelligent Systems (and AI in general) is that we do not learn how to learn how to build specific systems that behave intelligently, but rather how to understand the principles of how to build such systems. In other words, we try identify classes of problems with shared properties, and come up with generic solutions for those problems.

In order to make this possible, we are looking for shared representations of those problems. It turns out that it is very natural to represent many AI (decision) problems as State-Space problems, and that there are generic solutions to those problems using search algorithms.

This is what we are going to talk about in the first 4 lectures of this course, starting with the representations.

State space application

Robotic vacuum cleaners map a room with a state-space representation which is task specific. This mapping serves to simplify complex processes.



19

When an intelligent agent is in an environment, it has sensors through which it gains information about that environment. The input from these sensors can be transformed into a state space representation, which is a useful abstraction that helps the agent to make decisions.

Robotic vacuum cleaners follow this process by making detailed maps, or abstractions, of the homes they clean. These are further simplified into the concept of a room with or without dirt. By simplifying the environment to include only the information relevant to the problem, the vacuum only needs to decide whether to stay in the left room or move to the right room.

Looking at this example you will notice that there are different representations, and finding a good one (fitting the problem and the possible solution) is critical for building a good intelligent agent.

On the right hand side you see a simplified representation for a scenario with only two rooms (left and right). You see, there is no detailed representation of the corners and doors etc, because it does not matter for the specific problem we want to address, namely of deciding on moving between rooms and cleaning those rooms

(very high level descriptions of tasks).

Abstract problems → state space representations

A state space is an abstraction of the real world which serves to simplify a problem.

Real state → abstract state (graph)

Real action → abstract action

Real path that solves a real-world problem → abstract solution

One real world problem may have many alternative state-space representations.

We do this by using state-space representations, which are simply graphs, where the nodes are states the world can be in, and edges actions to get from one state of the world to another (note that we use the term “world” in a maybe unusual way, as it only describes the selected part of the real world that is relevant to our problem). We use Graphs to represent possible actions and states in a environment as they are mathematically well understood mathematical structures where we have an unambiguous vocabulary, and where the meeting of the notions is well understood.

Simplifying a real world problem means making specific decisions on how to represent the information relevant to the problem. As a result, there can be multiple, alternative state spaces that represent the same problem.

Problem solving agents

Given a goal, which states and actions should be considered?

- What are appropriate states and initial states?
- How to move between states?
- What is the cost of an action?

Goal formulation: what are the successful world states?

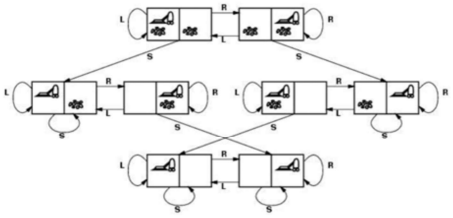
Search: what are possible action sequences to the goal? what is the best one?

Execute: perform the actions to reach the solution.

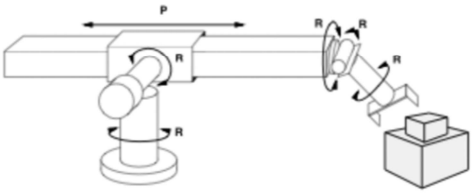
These are some of the questions associated with abstracting a real-world problem into a state space representation.

Problem solving agents must decide which state in the space is its goal state, how it will decide the sequence of actions to arrive at the goal state, and perform this sequence of actions.

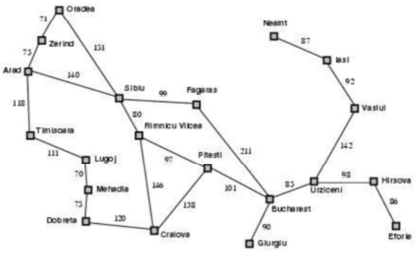
Examples



Vacuum World



Assembly Robot



Romanian Cities

22

Look at the graphical representation of the vacuum world example described previously. Imagine the intelligent vacuum starts in the state space in the top left of the graph. One possible action is to stay in its current position, represented by the looping edge. Another possible action is to move to the right room. Yet another is to clean the room it is currently in, shown by the edge leading to the left. This relatively small graph demonstrates how abstraction can make the vacuum world problem easy to oversee. The final state at the bottom of the graph represents both rooms being clean, at which point the vacuum can terminate.

We can abstract an autonomous assembly robot problem to concern only the angles at which each of its parts move, and whether or not there exists a cube that requires displacing. This can all be achieved through a vector representation.

Geographical maps can also be abstracted into graphs, as demonstrated with this graph of Romanian cities. The next slide details how this graph can be used to solve a problem.

State space representation of Romania

Concrete problem: depart from Bucharest, arrive in Arad

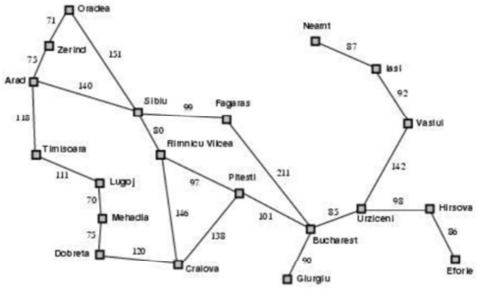
Formulation: states are cities (nodes), actions are flights between cities (edges)

Goal: Bucharest is the goal node

Solution: sequence of cities

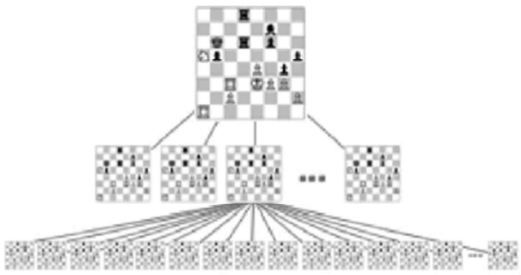
e.g. Arad, Sibiu, Fagaras, Bucharest

Execute: travel the sequence of nodes

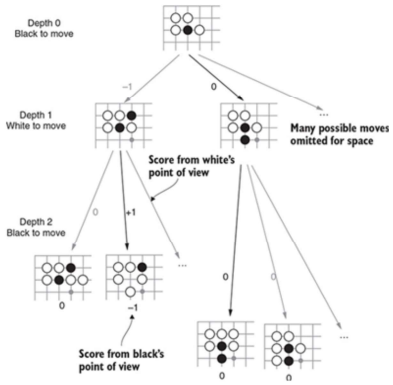


In this example of state space representation, the agent is a traveller, the states are Romanian cities, and the possible actions are traveling to the cities which are connected to the current state via an edge.

State space representation in games



Chess



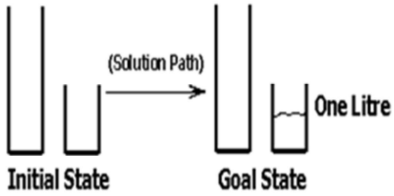
Go

Games may present a perfect environment for applying state space representations because they are discrete environments with a limited number of possible actions for every state. However, in complex games such as chess and Go, the graph representing all possible outcomes of a certain state becomes exponentially complex. Each transition between states represents a piece being placed on the board.

Let us practise

Let us model the following problem as a State-Space Representation problem

There are two jars with water, one with room for 3 litres, one for 5 litres. The goal is to get precisely 1 litre of water in one of the jars. You can fill a jar with fresh water, and empty it the other jar, or empty it fully.



Remember: Problem solving agents

Given a goal, which states and actions should be considered?

- What are appropriate states and initial states?
- How to move between states?
- What is the cost of an action?

Goal formulation: what are the successful world states?

Search: what are possible action sequences to the goal? what is the best one?

Execute: perform the actions to reach the solution.

These are some of the questions associated with abstracting a real-world problem into a state space representation.
Problem solving agents must decide which state in the space is its goal state, how it will decide the sequence of actions to arrive at the goal state, and perform this sequence of actions.

Part 4: Search Algorithms

Intelligent Systems
Vrije Universiteit

The following section concerns search algorithms. As we just saw with the example of graphs for games, using brute search methods to explore every node in a tree to predict which action will result in a win is impossible for the size of these graphs. Search algorithms help us navigate these graphs in a more efficient way.

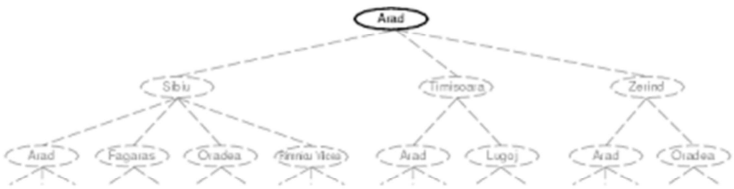
How do we solve these problems?

Searching the state space by generating a tree.

- Initial state is the tree root
- Nodes and leaves generated through successor functions
- Depth of a node is its distance from the root

Search methods work by expanding through the state space, starting at the initial state. The search algorithm will generate the tree according to the successor function which it employs.

Simple tree search



function **TREE-SEARCH(problem, strategy)** return a solution or failure

initialize search tree to the initial state of the problem

do

if no candidates for expansion then return failure

choose leaf node for expansion according to strategy

if node contains goal state then return solution

else expand the node and add resulting nodes to the search tree

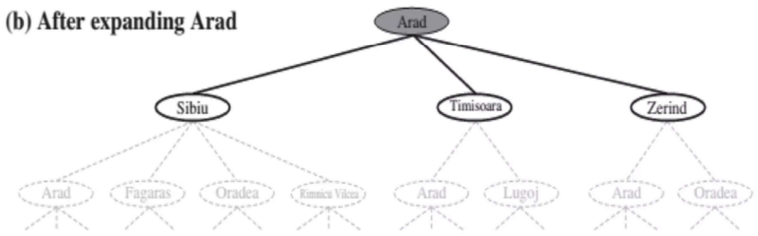
enddo

29

Above is the initialization phase of a simple tree search algorithm that generates a tree which represents the path the agent can take to get from the initial state to the goal state.

First we initialize the search tree as having the root node be the initial state, in this case the city of Arad. The children of the root node are all the states that the agent can possibly move to, in this case all cities to which there is a direct path from Arad. These are added to the tree. As long as the current node is not the goal node, this process will iterate for every new node that is added to the tree.

Simple tree search 2



function **TREE-SEARCH(problem, strategy)** return a solution or failure Initialize search tree to the initial state of the problem

do

if no candidates for expansion then return failure

choose leaf node for expansion according to strategy

if node contains goal state then return solution

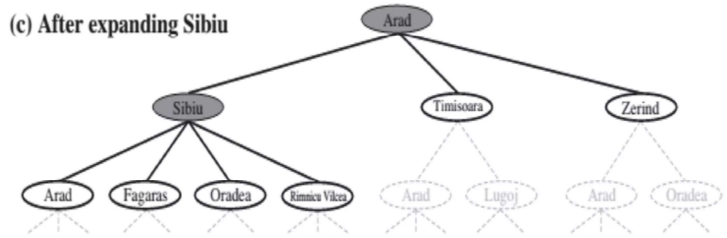
else expand the node and add resulting nodes to the search tree

enddo

30

This same process continues for each child node; the algorithm will continue to expand the tree until there are either no more nodes that can be added or the goal state has been reached. For these cases, algorithm will return failure or solution, respectively.

Simple tree search 3



function **TREE-SEARCH(problem, strategy)** return a solution or failure Initialize search tree to the initial state of the problem

do

if no candidates for expansion then return failure

choose leaf node for expansion according to strategy

if node contains goal state then return solution

else expand the node and add resulting nodes to the search tree # determines search process

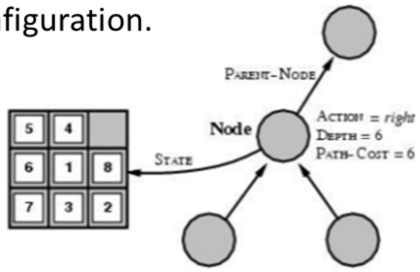
enddo

31

After Arad, the algorithm first expands Sibiu to all possible states that can be reached. Notice that Arad is repeated as one of the children of SIbiu, because it is in the list of possible successors. Our simple tree search algorithm isn't programmed to remember where it has been.

State space vs. search tree

A state is a representation of a physical configuration.



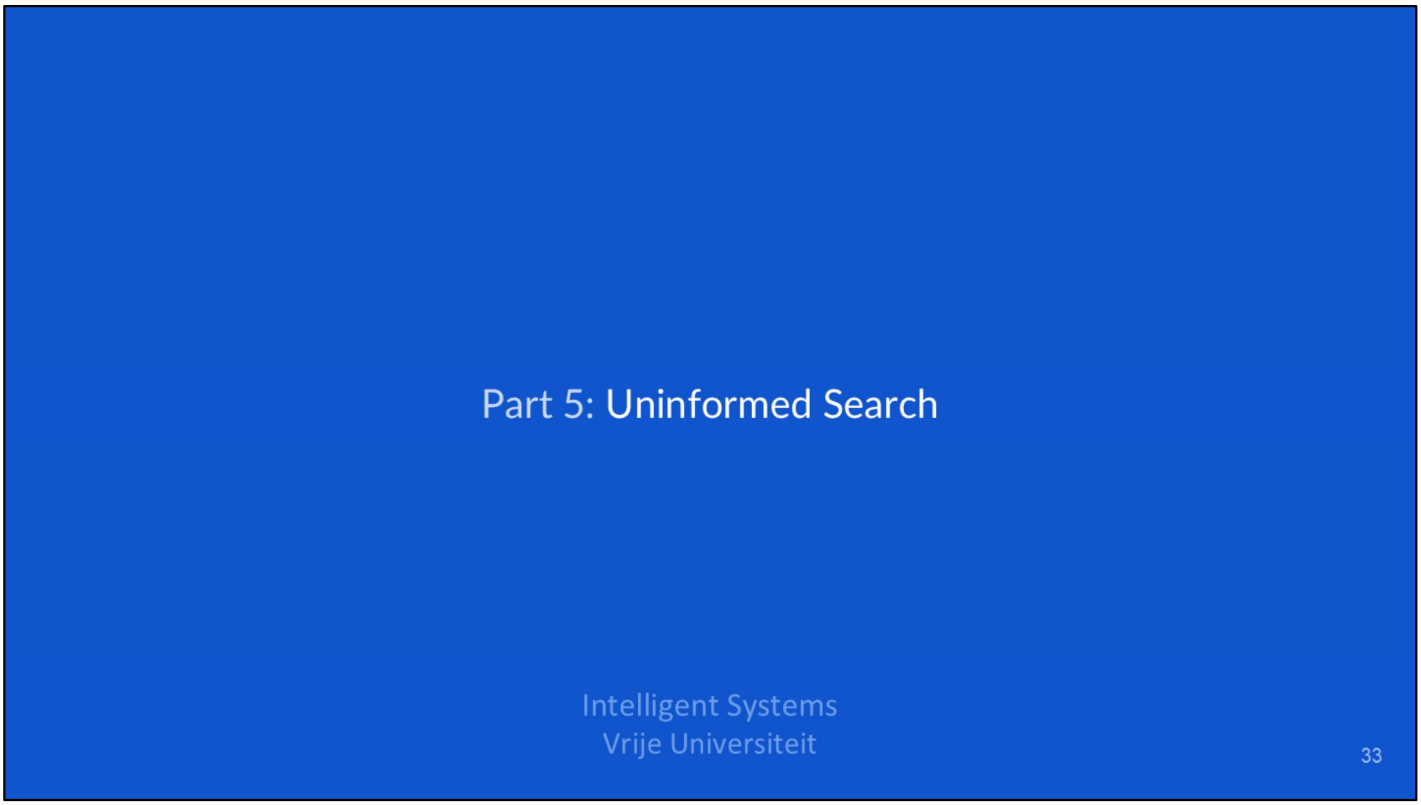
A node is a data structure belonging to a search tree.

A tree has parents and children, and includes path cost, and depth.

Node = <*state, parent-node, action, path-cost, depth*>

32

What is very important is this distinction between nodes, trees, and states. A state is a representation of the real world; the physical configuration in a model. Nodes correspond to states but are data structures that belong specifically to trees. This means that multiple nodes in the same tree can correspond to the same state. This leads us to an important question: which different strategies do we have for building trees that will lead us to the solution?



That brings us to uninformed search, which is a strategy for building these trees that requires little to no information.

Uninformed Search

Uninformed (blind) search uses only information which is available in the problem definition.

→ No strategy to determine one non-goal state is better than another.

→ Fringe contains generated nodes which are not yet expanded.

Expansion algorithms

- Breadth-first
- Uniform-cost
- Depth-first
- Depth-limited
- Iterative deepening
- Bidirectional

34

Uninformed search methods use limited to no information to build trees, without a strategy that determines if one non-goal state is better than another. For example, uninformed search has no preference amongst Sibiu, Timișoara, or Zerind as the successor of Arad.

A *fringe* is a data structure similar to a list, which stores all of the relevant nodes while the algorithm builds the tree. During the initialization phase, the fringe contains only the initial state, which is the root node.

Tree search algorithm

```
function TREE-SEARCH(problem,fringe) return a solution or failure
    fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
    loop do
        if EMPTY?(fringe) then return failure
        node REMOVE-FIRST(fringe)
        if GOAL-TEST[problem] applied to STATE[node] succeeds
                then return SOLUTION(node)
        fringe ← INSERT-ALL(EXPAND(node, problem), fringe)
    enddo
```

The EXPAND procedure determines the search strategy

35

After the initialization phase, we add the children of the root to the fringe. We loop through all of the nodes in the fringe and check (1) whether the fringe is empty, in which case we return failure because we didn't find a solution. (2) if the fringe is not empty, we remove the first node and check whether it is the goal node. If it is, (2a) then we return the solution. Otherwise, (2b) we expand the fringe to all possible successors or children of the current node.

Breadth-first search

Method: expand the shallowest unexpanded node

Implementation: *fringe* is a first-in-first-out (FIFO) queue

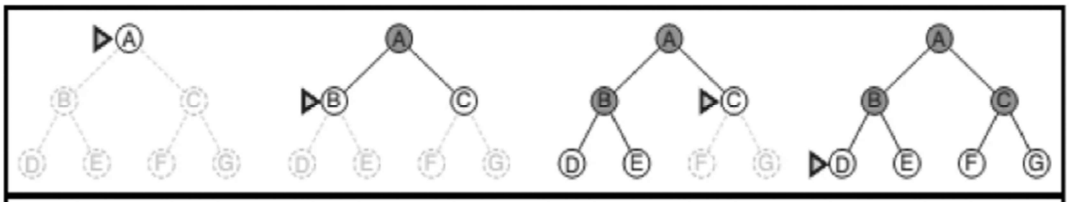


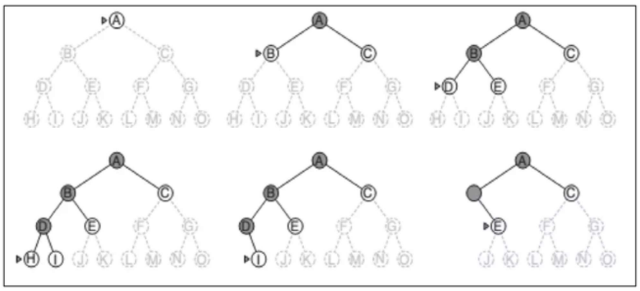
Figure 3.12 Breadth-first search on a simple binary tree. At each stage, the node to be expanded next is indicated by a marker.

There are different types of uninformed search, each characterized by the unique way in which their fringe operates. One type of uninformed search is called breadth-first search (BFS). When visualizing how BFS builds a tree, remember that it will always expand the shallowest unexpanded node from the current node. Looking at the tree, it moves left-to-right on every level before increasing its depth by 1 and adding all nodes at that level. BFS uses a FIFO queue. Imagine a line of people waiting outside a restaurant - the first people to stand in line are also the first people to leave when it is their turn.

Depth-first search

Method: expand the deepest unexpanded node

Implementation: *fringe* is a last-in-first-out (LIFO) stack



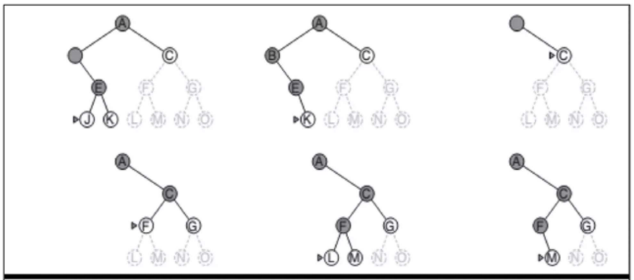


Figure 3.16 Depth-first search on a binary tree. The unexplored region is shown in light gray. Explored nodes with no descendants in the frontier are removed from memory. Nodes at depth 3 have no successors and M is the only goal node.

Depth-first search (DFS) expands the deepest possible node, prioritizing moving to a lower level before moving to the other nodes at the same level. Visually, it expands down the left side of the tree until it hits a dead end, it which case it will *backtrack* by one node and continue expanding one path further to the right. Its fringe is called a last-in-first-out (LIFO) stack. Imagine a stack of cafeteria trays - the first trays that entered the stack will be the last to leave because they are at the bottom.

Performance metrics

A strategy defines node expansion order.

Completeness	does the strategy always finds a solution if one exists?
Optimality	does it find the least-cost solution?
Time complexity	what number of nodes are generated/expanded?
Space complexity	what number of nodes are stored in memory during search?

Time and space complexity are measured in terms of

- ***b*** the maximum branching factor of the search tree
- ***d*** the depth of the least-cost solution
- ***m*** the maximum depth of the state space problem (may be ∞)

38

Now that we have looked at two different search methods, how do we evaluate whether they are good? How do we choose one search method over the other for our specific problem?

Four main performance metrics allow us to compare different search methods. Completeness refers to whether the strategy will always find a solution if one exists. For example, if the algorithm can get stuck in an infinite cycle, it may never terminate and thus be incomplete. Optimality refers to whether the solution found is the least-cost solution - think shortest path from root to goal. This will become more relevant once we explore cost functions. Time complexity refers to the number of nodes that were expanded before reaching the goal node. Note that this is not measured in milliseconds or other standard time metric because that can differ from computer to computer. Space complexity refers to the number of nodes that are stored in memory during search - think of nodes stored in the fringe.

To evaluate these algorithms mathematically we also define variables. *b*, the

maximum branching factor, is similar to the n-ary property explored earlier in this lecture. *d* is the depth of the least-cost solution found, and *m* is the maximum depth of the tree, which may be infinite.

Performance of uninformed search strategies

Performance Measure	Breadth-First	Depth-First
Completeness	*YES**	*NO*
Time	b^d	b^m
Space	b^d	bm
Optimality	*YES**	*NO*

*BFS finds the best solution when the branching factor is finite.

- ***b*** the maximum branching factor of the search tree
- ***d*** the depth of the least-cost solution
- ***m*** the maximum depth of the state space problem (may be ∞)

39

BFS is complete because it will always find a solution if one exists and optimal because it will find the shortest route there. This is because BFS moves systematically through each level of the tree, not skipping over any nodes as it expands. Time complexity is problematic because the number of nodes increases exponentially at each level. Space complexity is also problematic because the FIFO fringe must retain every single node at the current level.

DFS is incomplete because it is vulnerable to loops. Rather than systematically explore all nodes at one level, it will naively pick the first possible option. If we add the same state again at each level, the algorithm may cycle infinitely between two or more nodes. DFS is also not optimal because it may find a solution at depth 20 while there was an unexplored solution at depth 2 further to the right of our tree. This means it has not found the least-cost solution. Time complexity in DFS and BFS is comparable; in many cases they need to explore a similar number of nodes before reaching the goal. However, space complexity for DFS is significantly better because a LIFO stack does not need to retain all the nodes at the current level, because it

moves down a level before moving right. As such, its space complexity grows linearly rather than exponentially.

BFS evaluation: 2 lessons

(1) Memory requirement is a bigger problem than time complexity
(2) Exponential complexity search problems can't be solved by uninformed search methods except for the smallest instances

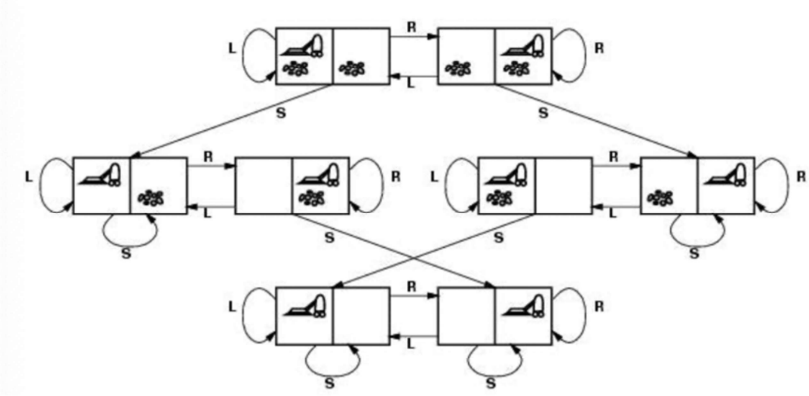
DEPTH2	NODES	TIME	MEMORY
2	110	0.11 ms	107kb
4	111100	11 milliseconds	106. Megabytes
6	10^7	11 seconds	1 gigabytes
8	10^9	2 mins	103 gb
10	10^{11}	3 min	10 terabytes
12	10^{13}	13 days	1 petabytes
14	10^{15}	3.5 years	99 petabytes

Figure 3.13 Time and memory requirements for breadth-first search. The numbers shown assume branching factor b = 10; 1 million nodes/second; 1000 bytes/node.

We've established that space and time complexity are problematic for BFS. Time complexity is less problematic due to the fact that we have very efficient computers these days. However, in a tree depth of 2 with 10 children, we have a branching factor of 10. As a result, at depth 2 we have 110 nodes, at depth 4 we have over 100 thousand. In a standard machine some time ago, that would translate to 100 megabytes of storage. For this problem, depth-first search is the solution.

DFS evaluation: cycles

DFS is incomplete because it is vulnerable to getting stuck in cycles.



41

Think about what may happen if we apply the depth-first search method to the Vacuum World example. Starting from the initial state, how can the algorithm get stuck in a loop? If this occurs, will the vacuum ever reach the goal state?

Search for Schnaptience

Schnaptience is a single-player Schnapsen variant.

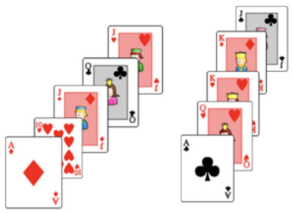
Game rules

Single-player game with 10 cards, two hands of 5 cards each.

Goal

Iteratively play a card that (1) follows suit or (2) has the same va

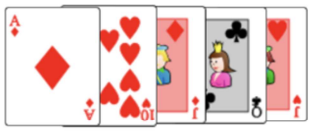
Goal-state reached when both hands are empty.

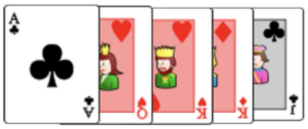


42

We can also apply these search methods to simple games without an adversary, such as Schnaptience. In this single-player Schnapps variant, the player puts down their cards such that each successor card has either the same suit or number as the most recent card. The *order* in which the cards are put down is analogous to a path through the Schnaptience search tree for a specific hand.

Schnaptience as State Space Problem





What is a state?

What is the initial state?

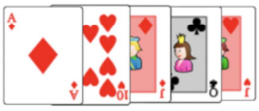
What is the goal state?

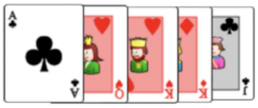
What are the actions?

Consider these questions. How can the two hands, the stack of cards, and the rules correspond to states and actions?

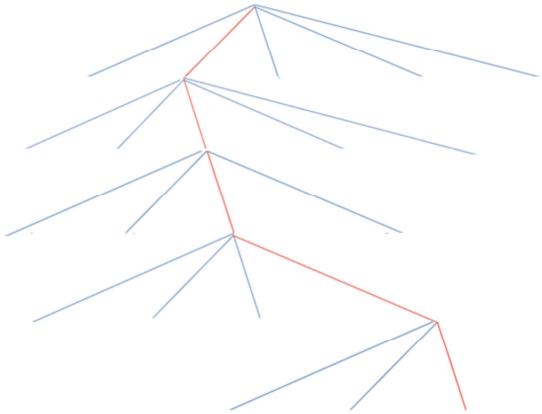
The state is the two hands of cards, with the initial state being all ten cards the player starts out with. The goal state is an empty hand, or no more cards left. An action is removing a card from the player's hand according to the rules of the game.

Schnaptience as State Space Problem





How many nodes maximally visited?
How many nodes in memory?



5^2 (each node has 5 more children - cards from other hand)

4^2

3^2

2^2

44

Let's build a tree. In their left hand, the player has five possible cards that can be played. If we expand by one level, there are only four possible cards. Then three, two, and finally only a single card that can be played.

If we multiple these together, we have 14,400 ($5^2*4^2*3^2*2^2*1^2$) possible nodes that can be explored for this incredibly simple game variant.

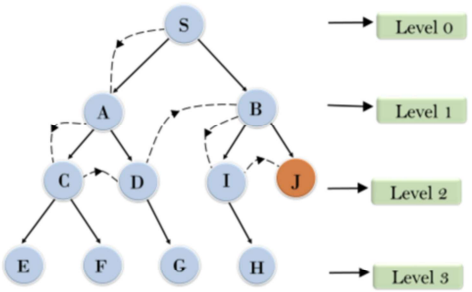
Depth-Limited search

Depth-limited search is depth-first search with limited depth *l*.

Method: expand the deepest unexpanded node, but nodes at depth *l* have no successors.

Implementation: If $l < d$ → incomplete search

If $l > d$ → not optim



DLS with ***l* = 2**

45

Depth-limited search solves one of the problems of depth-first search, namely its high space complexity. Depth-limited search starts with a predefined depth limit *l*, and performs depth-first search to depth *l* before backtracking. However, depth-limited search is still incomplete; the solution may be below the cutoff line that has been defined. This is a sort of *horizon effect* in which we can not see what is beyond the maximum depth, or the “horizon”.

Performance of Search Strategies

Performance Measure	Breadth-First	Depth-First	Depth-Limited
Completeness	*YES**	*NO*	*YES, if $l > d$*
Time	b^d	b^m	b^l
Space	b^d	bm	bl
Optimality	*YES**	*NO*	*NO*

*BFS finds the best solution when the branching factor is finite.

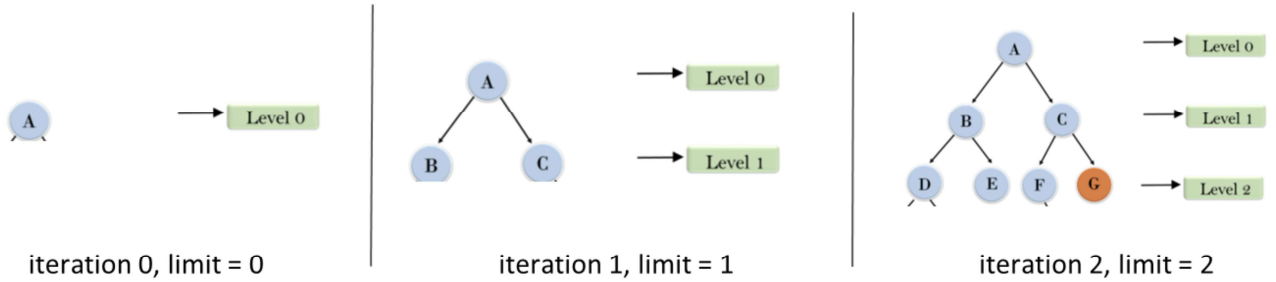
- ***b*** the maximum branching factor of the search tree
- ***d*** the depth of the least-cost solution
- ***m*** the maximum depth of the state space problem (may be ∞)

Let's return to our summary of performance for the algorithms we've looked at. Depth-limited has the same performance measures as depth-first, with a maximum depth of the state space *l*. Unlike depth-first search, depth-limited search is complete as long as the depth limit is greater than the depth of the solution.

Iterative Deepening Depth-First Search

Iterative deepening is a strategy to find the best depth limit *l*.

- Each consecutive search run adds one depth level
- Often used in combination with DFS
- Combines benefits of DFS and BFS



iteration 0, limit = 0 | iteration 1, limit = 1 | iteration 2, limit = 2

Iterative deepening is a search method which avoids the depth-first search vulnerability of getting stuck in infinite loops. It combines the benefits of depth-first and breadth-first search to combat the memory problem. This algorithm iteratively expands the search tree by a depth of 1 with every run until it finds the minimum depth to reach a goal state. Observe that each iteration throws away the tree that has been generated so far and starts from scratch.

Performance of Search Strategies

Performance Measure	Breadth-First	Depth-First	Depth Limited	Iterative Deepening
Completeness	*YES**	*NO*	*YES, if l > d*	*YES*
Time	b^d	b^m	b^l	b^d
Space	b^d	*bm*	*bl*	*bd*
Optimality	*YES**	*NO*	*NO*	*YES*

*BFS finds the best solution when the branching factor is finite.

- ***b*** the maximum branching factor of the search tree
- ***d*** the depth of the least-cost solution
- ***m*** the maximum depth of the state space problem (may be ∞)

48

As compared to the algorithms that we're now familiar with, iterative deepening performs better than depth-limited search because it is *optimal*. This means that there is a guarantee that the solution found is the one with the cheapest cost from initial state. The advantage of iterative deepening over breadth-first search, which is also optimal, is that iterative deepening uses significantly less memory in the process. How? Because iterative deepening throws away its entire memory after every unsuccessful iteration and systematically repeats it with a depth of +1.

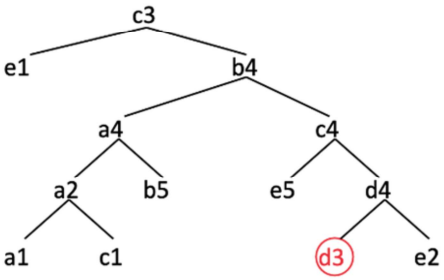
Part 6: Search Directions

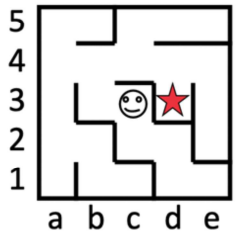
Intelligent Systems
Vrije Universiteit

Thus far we have been focused on reaching a goal state from some node in a tree; however, this is not always the smartest approach. In some situations, the information we have about the problem's goal and branching factor is an indicator that we should start with the goal and move backwards.

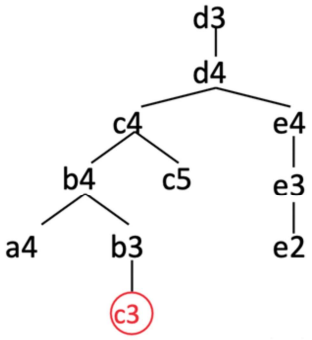
Search Directions

Data-driven starts at the current state.





Goal-driven starts at the goal state.



Search direction is a relative concept that depends on how you represent the state-space.

50

In the state-space search problems we have discussed so far, we took a **data-driven** approach, which means we start from the current state and reason backward to the goal state. Another possibility is to take a **goal-driven** approach in which we start at the goal state and reason our way forward to the goal state.

In this maze scenario, a data-driven tree starts from the current position of the agent at c3 and try to reach d3. Alternatively, we can represent that state-space such that the initial state d3 is the goal itself, and we reason towards the current state of c3.

Choosing a Search Direction

1. Is there a clear, distinct goal?
2. Which search direction results in a bigger branching factor?

Example Is Mark Rutte a descendant of Willem van Oranje?

Data-driven approach	**Goal-driven approach**
Check children, grandchildren, great grandchildren, etc. of Willem van Oranje:	Check parents, grandparents, great grandparents, etc. of Mark Rutte:
For *n* generations, 3 children per *n*: 3^n	For *n* generations, 2 parents per *n*: 2^n
3^{10} = *59049 branching factor*	2^{10} = *1024 branching factor*

51

There are two useful questions for deciding whether a data- or goal-driven approach is appropriate for the problem, (1) is there a clear, distinct goal and (2) which direction has a bigger branching factor. For example, given the problem of finding whether Mark Rutte is a descendant of Willem van Oranje, it is useful to visualise a family tree. A data-driven approach would have a branching factor of 3^n because we can imagine that there is a historical average of three children per set of parents since the life of Willem van Oranje. A goal-driven approach is more logical because it has a branching factor of only 2^n, because there are two parents for every child.

Summary & Homework

Summary

Real world problems can be represented in State Spaces

State Space problems can be solved through **search methods**

Tree search can be used to make decisions.

Uninformed search uses no or **partial** information.

BFS and **DFS** search broadly or deeply, respectively.

Performance measures include completeness, time and space complexity, optimality.

Homework

Practical assignment 1: graphs and trees → book slots with TAs

Work on theoretical material in work groups

Practice Schnapsen

Thank you for your attention and have a great day.